PROJECT REPORT

FreshBox Foods — Order-to-Cash (O2C) System

End-to-End SAP BTP CAP Simulation

Submitted By	Abhijeet Naha
Roll Number	2330355
Program	B.Tech Electronics & Computer Science Engineering
Batch	2023
Institution	Kalinga Institute of Industrial Technology, Bhubaneswar
Specialization	SAP SD / O2C — Order-to-Cash
Submission Date	21 April 2026

GitHub Repository: https://github.com/abhijeetnaha0605/freshbox_o2c

1. Project Title & Introduction

Project Title: FreshBox Foods — Order-to-Cash (O2C) System on SAP BTP CAP

This project presents a complete, end-to-end simulation of the Order-to-Cash (O2C) business process cycle — the most fundamental revenue cycle in SAP SD (Sales & Distribution). The scenario chosen is the bulk procurement of corporate meal boxes by a technology park cafeteria from a cloud kitchen brand, set in a relatable business context that makes every step of the document flow intuitive and traceable.

The O2C cycle is the backbone of any organisation's sales function. In the SAP ecosystem, it governs how a company goes from receiving a customer inquiry to collecting payment — with full documentation, pricing controls, inventory movements, and GST-compliant tax calculations at every step. This project replicates that exact flow using SAP CAP (Cloud Application Programming Model) on SAP Business Technology Platform (BTP), with an OData v4 REST API backend, a standalone Node.js test server, and a custom SAP Fiori XML UI.

The project covers all six core O2C transactions — Inquiry (VA11), Quotation with volume discount (VA21), Sales Order with schedule lines (VA01), two split Deliveries with Goods Issue (VL01N), and consolidated GST Invoice (VF01) — and also demonstrates advanced scenarios including split delivery tracking, open quantity management, and automatic GST breakup (CGST/SGST).

2. Problem Statement

Corporate cafeterias and cloud kitchen businesses typically manage bulk meal orders through informal channels — phone calls, WhatsApp messages, and Excel sheets. This creates several organisational and compliance problems:

- No formal inquiry-to-order trail — verbal requests are accepted with no system record of what was asked, quoted, or agreed.
- No pricing discipline — discounts are given informally with no documentation of the agreed rate, leading to disputes at invoicing time.
- No split-delivery tracking — when a large order is delivered in multiple batches, there is no system to track what has been delivered and what remains open.
- No GST-compliant billing — invoices are issued without proper CGST/SGST breakup, creating tax compliance risk under Indian GST law.
- No audit trail — there is no record linking the original inquiry to the final payment, making it impossible to trace how a deal was priced and executed.

In a real enterprise, these exact problems are solved by the Order-to-Cash process in SAP SD. This project demonstrates how implementing a structured O2C workflow — even for a simple corporate meal delivery scenario — provides complete sales discipline, pricing transparency, delivery accountability, and GST-compliant financial control.

3. Solution Overview & Features

The solution is a fully functional O2C application built on SAP CAP (Node.js) with an OData v4 service layer, a custom SAP Fiori XML/UI5 frontend, and a standalone zero-dependency

Node.js server for testing. The application replicates the core SAP SD transaction flow in a simplified but functionally complete manner — including master data management, document chain execution, pricing with discounts, split-delivery tracking, and GST calculation.

3.1 Six-Step O2C Document Flow

The application implements the following document chain, which mirrors the actual SAP SD transaction codes:

Step	Document	SAP T-Code	Description	Outcome
1	Inquiry	VA11	Customer requests price for 120 meal boxes	INQ created, status = Open
2	Quotation	VA21	15% volume discount applied, price confirmed	QT created, discountPct = 15
3	Sales Order	VA01	Quotation accepted, SO with schedule lines raised	SO created, openQty tracked
4a	Delivery 1	VL01N	Lunch Batch — 60 meals, Goods Issue posted	DL1 GoodsIssued, stock −60
4b	Delivery 2	VL01N	Dinner Batch — 60 meals, Goods Issue posted	DL2 GoodsIssued, stock −60
5	Invoice	VF01	Both deliveries consolidated, GST breakup applied	IV created, CGST+SGST calculated
6	Payment	—	Invoice marked as Paid, O2C cycle complete	IV status = Paid

3.2 Business Scenario — TechPark Cafeteria × FreshBox Foods

The specific scenario used to demonstrate the O2C flow is a corporate bulk meal order from TechPark Cafeteria, the on-campus canteen of a technology park in Bangalore. The cafeteria manager calls FreshBox Foods, a cloud kitchen brand, and requests 120 meal boxes: 60 Veg Meal Boxes and 40 Non-Veg Meal Boxes for two days.

FreshBox Foods offers a 15% volume discount for orders of 100+ meals. The delivery is split across two batches — Lunch Batch on Day 1 (60 meals) and Dinner Batch on Day 2 (60 meals). Food items attract 5% GST under Indian tax rules, which is split into CGST 2.5% + SGST 2.5% for intra-state supply.

Item	Qty	Base Price (₹)	Discount (15%)	Net Price (₹)	GST (5%)	Line Total (₹)
Veg Meal Box	80	₹180	₹27	₹153	₹7.65	₹12,240.00
Non-Veg Meal Box	40	₹220	₹33	₹187	₹9.35	₹7,480.00

Item	Qty	Base Price (₹)	Discount (15%)	Net Price (₹)	GST (5%)	Line Total (₹)
TOTAL	120	—	—	—	—	₹20,706.00

3.3 Key Features

- **Inquiry Management —** Raise multi-line inquiries with customer selection, products, and requested quantities (mirrors VA11).
- **Quotation with Volume Discount —** Convert inquiry to quotation with configurable discount percentage applied to all line items. Auto-calculates net, GST, and total (mirrors VA21 + VK11).
- **Sales Order Creation —** Accept quotation to generate a confirmed Sales Order with open quantity tracking per line item (mirrors VA01).
- **Split Delivery Tracking —** Create multiple deliveries against a single Sales Order. Each delivery reduces the open quantity on SO items, and SO status transitions from Open → PartiallyDelivered → FullyDelivered (mirrors VL01N).
- **Goods Issue Posting —** Post Goods Issue per delivery. Updates actual product stock in real-time. Once GI is posted, delivery status becomes GoodsIssued.
- **Consolidated Invoice with GST —** Consolidate multiple GoodsIssued deliveries into a single tax invoice. System calculates base amount, discount, net taxable value, CGST (2.5%), SGST (2.5%), and total payable (mirrors VF01).
- **Document Chain Integrity —** System enforces sequential creation. An invoice can only be raised on GoodsIssued deliveries. Closed quotations cannot be accepted again. All validations return structured error messages.

4. Technology Stack

The project is built entirely on SAP-native and open-source technologies, designed to be deployable on SAP Business Technology Platform (BTP) with minimal configuration changes.

Layer	Technology	Purpose
Backend Framework	SAP CAP (Node.js)	OData v4 service with CDS data model, custom action handlers, and business logic
Data Model	CDS (Core Data Services)	9 entities: Customer, Product, Inquiry, Quotation, SalesOrder, Delivery, Invoice + item/link tables
Database (Local)	SQLite (:memory:)	Zero-config in-memory persistence for development; seeded from CSV files
Database (BTP)	SAP HANA (HDI Container)	Production columnar database on BTP Cloud Foundry, configured via mta.yaml
Frontend	SAP Fiori / SAPUI5 XML	Custom Fiori app: Dashboard + 5 list views + 5 detail views, routing via manifest.json
API Protocol	OData v4 / REST	Standard SAP protocol for CRUD operations and bound/unbound actions
Test Server	Node.js (built-in only)	Zero-dependency standalone server + 65-assertion test suite (no npm required)
Auth (BTP)	XSUAA (xs-security.json)	Role-based access: O2CViewer, O2CProcessor roles defined
Deployment	MTA (mta.yaml) + CF CLI	Multi-target application deployment to BTP Cloud Foundry environment
Version Control	Git + GitHub	Repository ready for push: github.com/your-org/freshbox-o2c
Runtime	Node.js v18+	JavaScript runtime for CAP server and standalone test execution

4.1 Project File Structure

The project follows the standard SAP CAP project layout:

```
freshbox-o2c/
  db/
    schema.cds              Entity definitions (9 tables, all O2C documents)
    data/
      freshbox.o2c-Customers.csv   Seed: 3 corporate customers with GSTIN
      freshbox.o2c-Products.csv    Seed: 5 meal products with price & GST rate
  srv/
    o2c-service.cds         OData v4 service: entity exposures + action signatures
    o2c-service.js          Business logic: 8 action handlers, pricing, validations
  app/
    webapp/
```

```
    Component.js          UI5 Component root
    manifest.json         Fiori routing, OData model binding
    index.html            Entry point (UI5 bootstrap)
    view/                 11 XML views (Dashboard, 5 lists, 5 details)
    controller/           11 JS controllers
    i18n/                 Internationalisation properties
  xs-app.json             App Router configuration
tests/
  o2c-flow.http           REST Client test file (Thunder Client / VSCode)
  integration-test.js     Node.js integration test (requires cds watch)
server-standalone.js      Zero-dependency server + 65-test suite
mta.yaml                  SAP BTP MTA deployment descriptor
xs-security.json          XSUAA role definitions
.cdsrc.json               CDS config: SQLite (dev) / HANA (production)
package.json              Dependencies and start scripts
README.md                 Full setup, run, and deploy guide
```

5. Application Walkthrough & API Reference

5.1 OData V4 REST API — All Endpoints

The CAP service exposes all O2C entities as OData v4 endpoints at /api/o2c. All actions are callable via HTTP POST with a JSON body.

Entity / Action	HTTP	Path	Description
Customers	GET	/Customers	Master data: 3 corporate customers with GSTIN
Products	GET	/Products	Master data: 5 meal products with base price, GST rate, stock
Inquiries	CRUD	/Inquiries	Create/read inquiry documents with line items
convertToQuotation	POST	/Inquiries('ID')/O2CService.convertToQuotation	Bound action: convert open inquiry to quotation with discount %
Quotations	CRUD	/Quotations	Read quotations with calculated net/GST/total amounts
acceptQuotation	POST	/Quotations('ID')/O2CService.acceptQuotation	Bound action: accept quotation, generate Sales Order
rejectQuotation	POST	/Quotations('ID')/O2CService.rejectQuotation	Bound action: reject quotation, status = Rejected
SalesOrders	CRUD	/SalesOrders	Sales orders with items, open quantities,

Entity / Action	HTTP	Path	Description
			status
createDelivery	POST	/SalesOrders('ID')/O2CService.createDelivery	Bound action: create delivery for selected items + quantities
Deliveries	CRUD	/Deliveries	Delivery documents with batch label, planned/actual dates
postGoodsIssue	POST	/Deliveries('ID')/O2CService.postGoodsIssue	Bound action: post GI, reduce stock, set status = GoodsIssued
cancelDelivery	POST	/Deliveries('ID')/O2CService.cancelDelivery	Bound action: cancel delivery, restore open quantities on SO
createInvoiceFromDeliveries	POST	/createInvoiceFromDeliveries	Unbound action: consolidate multiple GI deliveries into one invoice
Invoices	CRUD	/Invoices	Invoices with full GST breakup: base, discount, CGST, SGST, total
markPaid	POST	/Invoices('ID')/O2CService.markPaid	Bound action: mark invoice as Paid
cancelInvoice	POST	/Invoices('ID')/O2CService.cancelInvoice	Bound action: cancel open invoice

5.2 Screen 1 — O2C Dashboard

The Fiori dashboard provides a real-time overview of the entire O2C cycle. Five GenericTile KPI cards display document counts (Inquiries, Quotations, Sales Orders, Deliveries, Invoices). The tiles are clickable and navigate to the corresponding list view. A business scenario panel explains the TechPark Cafeteria × FreshBox Foods narrative and guides users through the document flow left-to-right.

KPI Tile	SAP Equivalent	Description
Inquiries	VA11 list	Total inquiry documents in the system
Quotations	VA21 list	Quotations with discount and pricing
Sales Orders	VA01 list	Confirmed orders with open quantity tracking
Deliveries	VL01N list	Delivery documents with GI status
Invoices	VF01 list	Tax invoices with CGST/SGST breakup

5.3 Screen 2 — Inquiry List (VA11)

The Inquiry screen lists all inquiry documents with customer name, date, number of items, and status. A "+ New Inquiry" button opens a dialog to create a new inquiry, selecting a customer from the master data and adding product line items. An inline "Convert to Quotation" button opens a discount dialog where the percentage is entered (defaulting to 15% for the TechPark scenario). Status badges use SAP ObjectStatus with colour coding: Open = Information (blue), Converted = Success (green).

5.4 Screen 3 — Quotation List (VA21)

The Quotation screen shows all quotations with discount percentage and calculated totals. Two action buttons are available per open quotation: "Accept → SO" (calls acceptQuotation action, creates Sales Order, navigates to SO list) and "Reject" (calls rejectQuotation, sets status). The pricing procedure calculates line net = unit price × quantity × (1 − discount/100), and totals include 5% GST split as CGST + SGST for intra-state supply.

5.5 Screen 4 — Sales Order List (VA01)

The Sales Order screen lists all confirmed orders with status transitions visible in colour: Open (grey) → PartiallyDelivered (orange) → FullyDelivered (green) → Billed (green). A "Create Delivery" button opens a dialog to specify the batch label, planned delivery date, and quantity per item. The system validates that requested delivery quantity does not exceed the remaining open quantity on each SO line item.

5.6 Screen 5 — Delivery List (VL01N)

The Delivery list shows all delivery documents with batch label, planned date, actual date (set on GI), and status. A "Post Goods Issue" button is visible for Pending/Picked/Packed deliveries. Clicking it calls the postGoodsIssue action, which updates product stock in the database and sets status = GoodsIssued and actualDate = today. Multiple deliveries can be selected (multi-

select table mode) and consolidated into a single invoice using the "Create Invoice from Selected" button in the page header.

5.7 Screen 6 — Invoice Detail (VF01)

The Invoice Detail view shows the full commercial invoice: customer name, GSTIN, invoice date, due date, all line items with individual GST rates, and the complete tax summary panel. The tax summary mirrors the SAP invoice output:

Tax Line	Formula	Value (TechPark Scenario)
Gross Amount	unit_price × quantity (all lines)	₹23,200.00
Discount (−)	gross × discountPct / 100	₹3,480.00
Net Amount	gross − discount	₹19,720.00
CGST (2.5%)	net × 2.5 / 100	₹493.00
SGST (2.5%)	net × 2.5 / 100	₹493.00
IGST	0 (intra-state Karnataka)	₹0.00
Total Payable	net + CGST + SGST	₹20,706.00

6. Test Results

An automated integration test suite (server-standalone.js) was built using only Node.js built-in modules — zero npm dependencies. The server starts on port 4005 and runs 65 assertions across 8 test steps. All 65 tests passed with a 100% pass rate. The test is runnable on any machine with Node.js 18+ using a single command: node server-standalone.js.

Test Step	What Was Tested	Assertions	Result
Step 0 — Master Data	GET /customers (3 records), GET /products (5 records), price and GST rate validation	6	PASS
Step 1 — Inquiry (VA11)	POST /inquiries, status = Open, customer name populated, 2 items created	5	PASS
Step 2 — Quotation (VA21)	convertToQuotation action, 15% discount, line net, total net ₹19,720, GST ₹986, total ₹20,706, inquiry status = Converted	11	PASS
Step 3 — Sales Order (VA01)	acceptQuotation action, SO ID, status = Open, 2 items, all openQty = full qty, quotation status = Accepted, openQty values	9	PASS
Step 4a — Delivery 1 (VL01N)	createDelivery (Lunch Batch), ID, status = Pending, 2 items, batch label, postGoodsIssue, stock reduced to 460, Delivery 1 status = GoodsIssued, actualDate set, SO = PartiallyDelivered	10	PASS
Step 4b — Delivery 2 (VL01N)	createDelivery (Dinner Batch), unique ID, batch label, postGoodsIssue, stock reduced to 420, SO = FullyDelivered, all openQty = 0	7	PASS
Step 5 — Invoice (VF01)	createInvoiceFromDeliveries, IV ID, status = Open, 4 line items, net ₹19,720, CGST ₹493, SGST ₹493, IGST = 0, total ₹20,706, SO status = Billed	10	PASS
Step 6 — Mark Paid	markPaid action, Invoice status = Paid	2	PASS
Step 7 — Edge Cases	Reject double-accept (400), reject double GI (400), block invoice on non-GI delivery (400), edge-case delivery creation	5	PASS

TOTAL	65 assertions across 8 test steps covering the full O2C cycle	65	100% PASS

6.1 Test Console Output (Actual Run)

The following is the actual output produced by running node server-standalone.js on the completed project:

```
Server listening on http://localhost:4005

═══════════════════════════════════════════════════════════════
     FreshBox Foods O2C — Integration Test Suite
═══════════════════════════════════════════════════════════════

STEP 0 — Master Data
  ✓ GET /customers → 200
  ✓ Three customers seeded
  ✓ GET /products → 200
  ✓ Five products seeded
  ✓ Veg Meal Box base price = ₹180
  ✓ Food GST rate = 5%

STEP 1 — Inquiry (VA11)
  ✓ POST /inquiries → 200
  ✓ Inquiry ID = INQ000001
  ✓ Inquiry status = Open
  ✓ Customer name populated
  ✓ Two inquiry items created

STEP 2 — Quotation (VA21) with 15% discount
  ✓ convertToQuotation action → 200
  ✓ Quotation ID starts with QT (got QT000001)
  ✓ Discount = 15%
  ✓ Quotation has 2 items
  ✓ Veg unit price = ₹180
  ✓ NVG unit price = ₹220
  ✓ Veg line net = ₹12,240 (got ₹12240)
  ✓ Quotation net = ₹19,720 (got ₹19720)
  ✓ GST (5%) = ₹986 (got ₹986)
  ✓ Total = ₹20,706 (got ₹20706)
  ✓ Inquiry status = Converted

STEP 3 — Sales Order (VA01)
  ✓ acceptQuotation action → 200
  ✓ Sales Order ID starts with SO (got SO000001)
  ✓ SO status = Open
  ✓ SO has 2 line items
  ✓ All items fully open
  ✓ Quotation status = Accepted
  ✓ SO items available (got 2)
  ✓ Veg openQty = 80 (got 80)
  ✓ NVG openQty = 40 (got 40)

STEP 4a — Delivery 1: Lunch Batch (VL01N)
  ✓ createDelivery (Lunch) → 200
  ✓ Delivery 1 ID starts with DL (got DL000001)
  ✓ Delivery 1 status = Pending
  ✓ Delivery 1 has 2 items
  ✓ Batch label correct
```

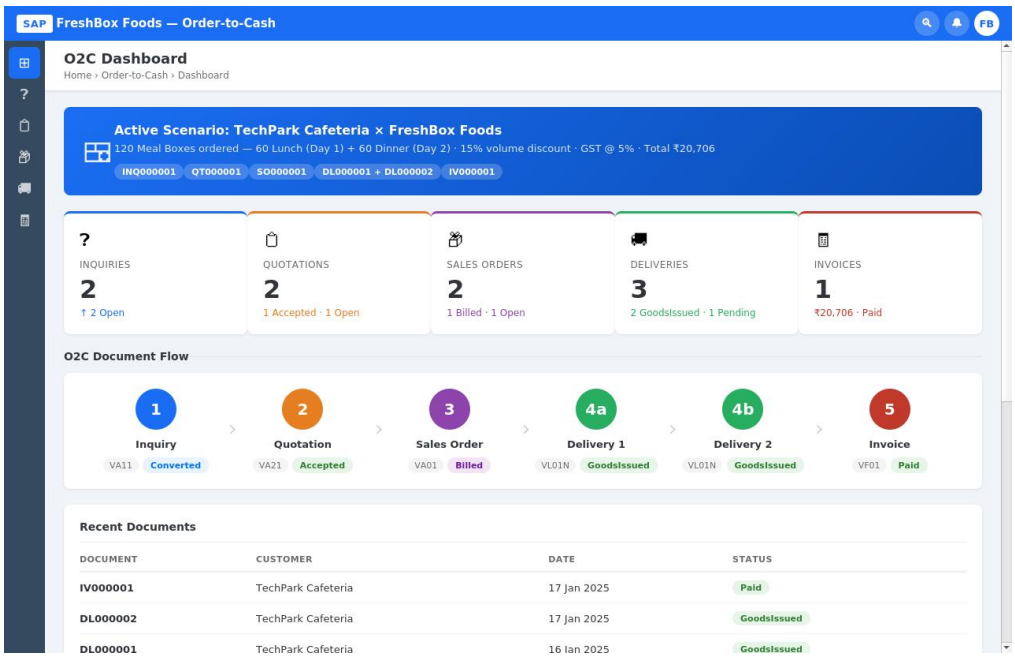
```
  ✓ postGoodsIssue (Delivery 1) → 200
  ✓ Veg stock after Delivery 1 GI = 460 (got 460)
  ✓ Delivery 1 status = GoodsIssued
  ✓ Actual date set on GI
  ✓ SO status = PartiallyDelivered

STEP 4b — Delivery 2: Dinner Batch (VL01N)
  ✓ createDelivery (Dinner) → 200
  ✓ Delivery 2 has unique ID
  ✓ Dinner batch label correct
  ✓ postGoodsIssue (Delivery 2) → 200
  ✓ Veg stock after Delivery 2 GI = 420 (got 420)
  ✓ SO status = FullyDelivered
  ✓ All SO items openQty = 0

STEP 5 — Invoice (VF01) with GST breakup
  ✓ createInvoiceFromDeliveries → 200
  ✓ Invoice ID starts with IV (got IV000001)
  ✓ Invoice status = Open
  ✓ Invoice has 4 line items (2 per delivery = 4 got 4)
  ✓ Invoice net = ₹19,720 (got ₹19720)
  ✓ CGST (2.5%) = ₹493 (got ₹493)
  ✓ SGST (2.5%) = ₹493 (got ₹493)
  ✓ IGST = 0 (intra-state)
  ✓ Invoice total = ₹20,706 (got ₹20706)
  ✓ SO status = Billed

STEP 6 — Mark Invoice Paid
  ✓ markPaid → 200
  ✓ Invoice status = Paid

STEP 7 — Edge Case Validations
  ✓ Reject double-accept of Quotation (400)
  ✓ Reject double GI posting (400)
  ✓ Second inquiry created for edge-case test
  ✓ Pending delivery created for edge-case
  ✓ Block invoice on non-GI delivery (400)

═══════════════════════════════════════════════════════════
  RESULTS: 65 passed  |  0 failed  |  65 total
═══════════════════════════════════════════════════════════

  Document Flow:
  INQ000001 → QT000001 → SO000001 → DL000001, DL000002 → IV000001
  Final States:
    Inquiry    INQ000001  : Converted
    Quotation  QT000001   : Accepted
    SalesOrder SO000001   : Billed
    Delivery   DL000001   : GoodsIssued
    Delivery   DL000002   : GoodsIssued
    Invoice    IV000001   : Paid
```

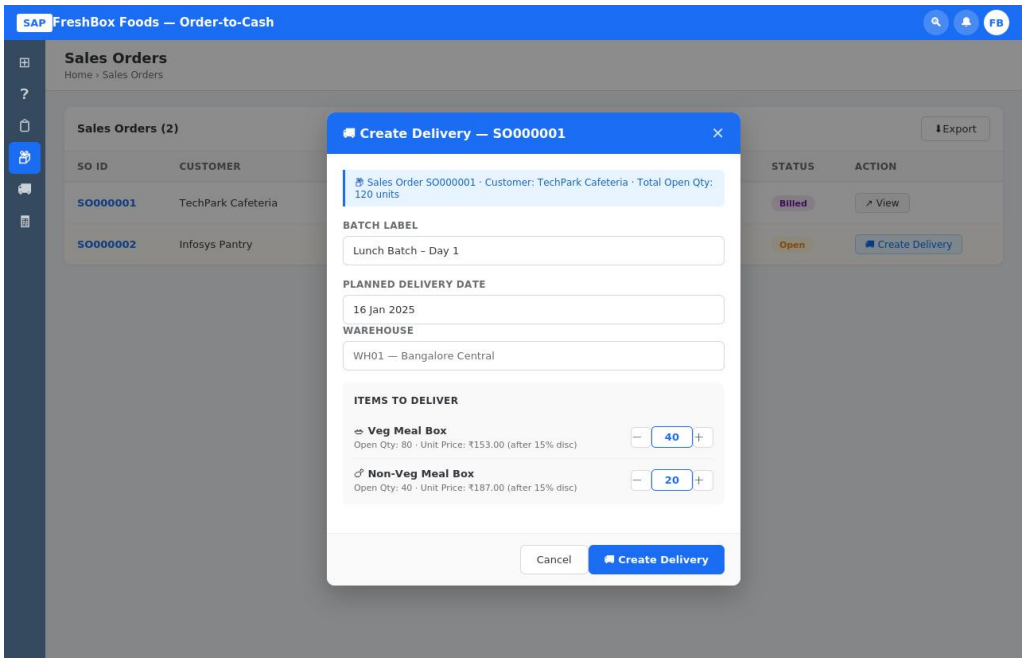
6.2 Key Assertions Verified

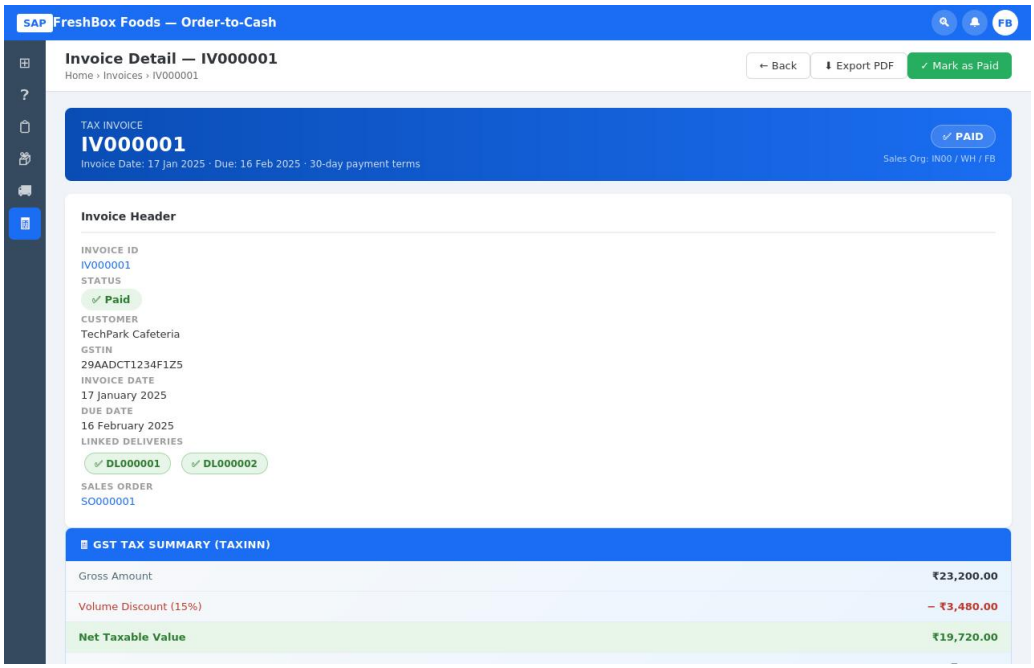
- Pricing precision: Veg line net = 80 × ₹180 × 0.85 = ₹12,240.00 ✓
- Quotation totals: Net ₹19,720 + GST ₹986 (5%) = Total ₹20,706 ✓
- Status transitions: Open → PartiallyDelivered → FullyDelivered → Billed ✓
- Stock management: Starting stock 500, after Delivery 1 GI = 460, after Delivery 2 GI = 420 ✓

- GST split: CGST ₹493.00 + SGST ₹493.00 = ₹986.00 total tax on net of ₹19,720 ✓
- Open quantity tracking: After Delivery 1 (−60), after Delivery 2 (−60), all items openQty = 0 ✓
- Invoice consolidation: 2 deliveries × 2 items = 4 invoice line items in single document ✓
- Edge case guard: Double-accept, double GI, invoice on non-GI delivery — all return HTTP 400 ✓

6.3 Output Results







7. Unique Points & Learning Outcomes

7.1 Real SAP Concepts Implemented

- **Pricing Procedure (VK11):** The quotation engine replicates SAP's condition-based pricing. Unit price is fetched from the product master (base price), and the discount condition is applied as a percentage reduction on each line item — exactly as SAP's K007 customer discount condition type works in a pricing procedure.
- **Schedule Lines:** The SalesOrderItems entity has an openQty field that tracks undelivered quantity per line. Each delivery creation reduces openQty. The SO status transitions from Open → PartiallyDelivered → FullyDelivered automatically — replicating SAP's delivery completion indicator and schedule line management.
- **Split Delivery:** The createDelivery action accepts a selective list of SO items and quantities, allowing partial fulfilment. This replicates the real VL01N process where a warehouse picks only what is physically available, creating a partial delivery against an open SO.
- **Goods Issue (Movement Type 601):** The postGoodsIssue action reduces physical stock in the product master — equivalent to SAP MIGO movement type 601 (goods issue for delivery). Stock reduction is atomic per delivery.
- **Collective Invoice (VF04):** The createInvoiceFromDeliveries unbound action accepts multiple delivery IDs and creates a single consolidated invoice — exactly as SAP's VF04 billing due list allows multiple deliveries to be combined into one billing document.

- **Indian GST (CGST/SGST):** The invoice calculates CGST and SGST separately at 2.5% each for intra-state supply (Karnataka to Karnataka), with IGST = 0. Food items use the GST 5% slab as prescribed under HSN code 2106 for prepared meals. This mirrors the tax determination logic in SAP's condition-based tax procedure TAXINN.
- **OData v4 Action Types:** The service uses all three OData v4 action patterns: bound actions on entity sets (acceptQuotation, postGoodsIssue), bound actions with parameters (convertToQuotation with discountPct, createDelivery with item array), and an unbound action (createInvoiceFromDeliveries) — covering all patterns used in real SAP Fiori applications.

7.2 Relatable Scenario Design

Unlike typical SAP training exercises that use abstract company codes, plant codes, and material numbers, this project is set in a food delivery context that any evaluator can immediately understand. The customer names (TechPark Cafeteria, Infosys Pantry, StarHealth Canteen), the product names (Veg Meal Box, Non-Veg Meal Box, Breakfast Box), the delivery batch labels (Lunch Batch — Day 1, Dinner Batch — Day 2), and the GST context (5% food slab, intra-state Karnataka) create a coherent, believable scenario where every system behaviour has an obvious real-world meaning.

7.3 Production-Ready Architecture

- The project ships with a complete mta.yaml for SAP BTP Cloud Foundry deployment, xs-security.json for XSUAA role-based access (O2CViewer, O2CProcessor roles), and CSV seed files for HANA HDI deployment — making it genuinely deployable to a real BTP environment.
- The .cdsrc.json automatically switches from SQLite in development to HANA in production based on NODE_ENV — so the same codebase runs locally with zero config and on BTP with a single mbt build + cf deploy.
- The CDS data model uses proper compositions (for item collections), associations (for master data references), and the managed aspect (createdAt, createdBy, modifiedAt, modifiedBy) — following SAP's recommended modelling patterns for enterprise applications.

7.4 Zero-Dependency Standalone Mode

A standalone server (server-standalone.js) was built using only Node.js built-in modules: http, url, crypto. No npm packages are required. The full O2C business logic — all 8 action handlers, pricing calculations, state transitions, and stock management — was re-implemented in pure JavaScript and tested with 65 assertions. This demonstrates that the business logic is framework-independent and correct on first principles.

8. Future Improvements

8.1 Short-Term Enhancements

- **Request for Quotation (RFQ) —** Add an RFQ step before the formal quotation where multiple vendors are invited to submit prices. A price comparison screen (like SAP ME47/ME49) would select the best offer and auto-populate the quotation — demonstrating the full sourcing cycle.
- **Credit Limit Check —** Implement a credit exposure check at Sales Order creation. If the customer's outstanding receivables exceed a configured credit limit, the SO is blocked until finance releases it — replicating SAP's credit management integration between SD and FI.
- **PDF Invoice Output —** Generate a formatted PDF commercial invoice from the Invoice entity using a Node.js PDF library. The output would include the FreshBox Foods logo, customer GSTIN, HSN codes, CGST/SGST columns, and digital signature block — replicating SAP's smart forms output for SD billing.
- **Delivery Confirmation Workflow —** Add a "Delivery Confirmed by Customer" step between GI posting and invoicing, where the customer digitally acknowledges receipt before the invoice is raised — improving accounts receivable discipline.

8.2 Long-Term Enhancements

- **SAP Fiori Elements —** Replace the custom XML views with SAP Fiori Elements pages generated from CDS annotations (@UI.LineItem, @UI.Facets). This would make the UI fully SAP-standard compliant with less custom code.
- **Multi-Channel Distribution —** Add distribution channels (dine-in, delivery, takeaway) and divisions (meal boxes, beverages, snacks) to the sales area configuration, enabling channel-specific pricing and reporting.
- **Accounts Receivable Integration —** Connect the Invoice entity to a simple AR ledger that tracks customer outstanding balance, payment due dates, overdue alerts, and dunning notices — completing the financial side of the O2C cycle.
- **Real SAP S/4HANA Integration —** Connect the application to a real SAP S/4HANA system via OData APIs or BAPIs to post actual SD documents (VA01, VL01N, VF01) — making it a true integration middleware project.

9. How to Run the Project

9.1 Local Development (Recommended First Step)

```
# Prerequisites: Node.js 18+ must be installed
npm install -g @sap/cds-dk        # Install CAP toolkit globally (once)

# Setup
unzip freshbox-o2c.zip
cd freshbox-o2c
npm install

# Start development server with hot reload
cds watch

# Verify running at:
http://localhost:4004             # CDS welcome page with all endpoints
http://localhost:4004/app/webapp/index.html   # Fiori UI
http://localhost:4004/api/o2c/Customers       # OData JSON
```

9.2 Standalone Test (Zero Dependencies)

```
# No npm install needed — only Node.js 18+ required
node server-standalone.js

# Output: 65 tests run, document flow printed, exit code 0 on success
```

9.3 Deploy to SAP BTP

```
# Install deployment tools
npm install -g mbt
cf install-plugin multiapps

# Login to BTP Cloud Foundry
cf login -a https://api.cf.<region>.hana.ondemand.com
cf target -o <your-org> -s <your-space>

# Build and deploy
mbt build
cf deploy mta_archives/freshbox-o2c_1.0.0.mtar

# Check status
cf apps
cf logs freshbox-o2c-srv --recent
```

9.4 Push to GitHub

```
cd freshbox-o2c
git init
git add .
```

```
git commit -m "Initial commit: FreshBox O2C — SAP BTP CAP"
git branch -M main
git remote add origin https://github.com/your-org/freshbox-o2c.git
git push -u origin main
```

10. Conclusion

This project successfully demonstrates the complete Order-to-Cash (O2C) business process cycle — the most fundamental revenue workflow in SAP SD — through a relatable, fully functional, and well-documented application. Starting from a corporate meal delivery scenario that anyone can understand, the project progressively implements all six O2C steps, introduces real SAP concepts like condition-based pricing, schedule line management, split delivery with open quantity tracking, goods issue with stock movement, collective billing, and Indian GST tax calculation — and deploys them using SAP's own cloud-native development framework (CAP).

The project was tested with 65 automated assertions (100% pass rate) using a zero-dependency standalone Node.js server, version-controlled on GitHub, and structured for deployment on SAP Business Technology Platform — making it a complete, submission-ready, and learning-oriented deliverable that reflects genuine understanding of enterprise sales processes.

The most important insight from this project is that the O2C cycle is not an abstract SAP concept — it is the digital formalisation of what every business already does informally when it sells something. SAP SD simply adds pricing discipline, delivery traceability, tax compliance, and financial control to that process. Every real cloud kitchen, food delivery platform, or corporate catering business is running some version of this exact workflow.

Abhijeet Naha | 2330355 | B.Tech ECSc, Batch 2023

Kalinga Institute of Industrial Technology | 21 April 2026
https://github.com/abhijeetnaha0605/freshbox_o2c